INTERACTIVE QUIZ APPLICATION

A PROJECT REPORT

SUBMITTED BY

Roll Number	Registration Number	Student Code	Student Name
26102410028	26102000150	BWU/MCA/26/032	Sumi Bera
26102410027	26102000149	BWU/MCA/26/031	Suvho Dhara
26102410040	26102000162	BWU/MCA/26/045	Swati Madduney
26102410037	26102000159	BWU/MCA/26/041	Pranay Jana
26102410039	26102000161	BWU/MCA/26/044	Apurba Kuiti
26102410035	26102000157	BWU/MCA/26/039	Jeet Sarkar

In partial fulfillment for the award of the degree

MASTER OF COMPUTER APPLICATIONS

IN

DEPARTMENT OF COMPUTATIONAL SCIENCES



BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to the Management and Faculty of **Brainware University** for providing us with the opportunity, guidance, and resources required to successfully complete this project. Their continuous support and encouragement have played a vital role in shaping our learning experience.

We extend our sincere thanks to our esteemed project supervisor, **Mr. Sourav Sandhukhan, Assistant Professor**, whose constant guidance, valuable suggestions, and insightful feedback have been instrumental throughout the development of this project. His expertise, patience, and motivation helped us overcome challenges and understand the core concepts effectively.

We are also deeply thankful to **Dr.Jayanta Aich, Head of the Department of Computational Sciences**, for providing a supportive academic environment and the necessary facilities that enabled us to complete this work successfully.

Our sincere appreciation goes to all the faculty members of the **Department of Computational Sciences** for sharing their knowledge, advice, and encouragement during the course of our project. Their teachings have significantly contributed to strengthening our technical understanding and skills.

Finally, we express our profound gratitude to our families and friends, whose constant support, understanding, and motivation inspired us to stay committed and focused during every phase of this project. Without their encouragement, this achievement would not have been possible.

SIGNATURE:--

Sumi Bera
(BWU/MCA/26/032)

Suvho Dhara
(BWU/MCA/26/031)

Swati Madduney
(BWU/MCA/26/045)

Pranay Jana
(BWU/MCA/26/041)

Apurba Kuiti
(BWU/MCA/26/044)

Jeet Sarkar
(BWU/MCA/26/039)

ABSTRACT

The **Quiz Application** is a web-based application developed as a group project for the Web Designing course. The main objective of this project is to provide an interactive and user-friendly platform where users can participate in quizzes and test their knowledge on different topics.

The application is designed using web technologies such as **HTML, CSS, and JavaScript** to create an attractive interface and provide interactive functionality. Users can start a quiz, select answers from multiple-choice questions, navigate through questions, and view their final score after completing the quiz. The application provides instant feedback and makes the learning process more engaging and enjoyable.

The Quiz Application demonstrates the practical implementation of web designing concepts such as **responsive design, user interface design, navigation, form handling, JavaScript logic, and dynamic content manipulation**. The project aims to combine education and entertainment while improving users' knowledge and problem-solving skills through an easy-to-use digital platform.

INDEX

- ACKNOWLEDGMENT
- ABSTRACT
- LIST OF FIGURE
- LIST OF TABLE
- LIST OF SCREENSHOTS

TITLE	PAGE NO
1. INTRODUCTION	1
2. LITERATURE SURVEY	2
3. SYSTEM ANALYSIS AND DESIGN	3 - 5
4. IMPLEMENTATION	6 - 7
5. OUTPUT SCREENSHOT	5 - 10
6. SOURCE CODE	11 - 14
7. TESTING AND DEBUGGING	15 - 16
8. CONCLUSION AND FUTURE ENHANCEMENT	17
9. REFERENCES	18

1. INTRODUCTION

In today's digital era, technology has transformed the way people learn, communicate, and access information. Web-based applications have become an important part of education because they provide users with easy access to learning resources from anywhere and at any time. Among various digital learning tools, online quiz applications are an effective way to make the learning process more interactive, interesting, and engaging.

A **Quiz Application** is a web-based application designed to allow users to participate in quizzes and evaluate their knowledge through a series of questions. It provides a simple platform where users can read questions, select answers from multiple options, and receive their final score after completing the quiz. Such applications can be useful for students, teachers, educational institutions, and anyone who wants to practice or test their knowledge.

The **Quiz Application** is developed as a group project for the **Web Designing course**. The project focuses on applying the concepts and techniques learned during the course to develop a functional and attractive website. The application is designed with a user-friendly interface so that users can easily understand the instructions, attempt questions, and view their results without difficulty.

The project uses **HTML, CSS, and JavaScript** as the primary technologies. HTML is used to create the structure and content of the web pages, including headings, buttons, questions, and answer options. CSS is used to design the interface, control layouts, add colors, fonts, spacing, animations, and make the application visually appealing. JavaScript is used to add functionality and interactivity, such as displaying questions, checking selected answers, calculating scores, navigating between questions, and displaying the final result.

The project demonstrates important web designing concepts such as **web page structure, responsive layouts, navigation, buttons, forms, styling, user interaction, event handling, and dynamic content manipulation**. Working on the project as a group also helps students understand teamwork, task distribution, communication, and project management.

The Quiz Application can be designed to support different categories and levels of quizzes. For example, quizzes can be created for subjects such as **Web Designing, Computer Science, General Knowledge, Mathematics, Science, and other educational topics**. Different difficulty levels can also be introduced to make the application suitable for users with different levels of knowledge.

Overall, the **Quiz Application** is a practical implementation of web designing concepts that combines an attractive user interface with interactive functionality. The project aims to make quizzes more convenient, engaging, and efficient while providing the development team with valuable hands-on experience in designing and developing a web-based application.

2. LITERATURE SURVEY

The development of web-based quiz applications has gained significant importance with the growth of online education and digital learning platforms. Online quizzes provide an effective method for evaluating knowledge, improving learning outcomes, and providing immediate feedback to users. Various educational websites and learning management systems use quiz-based assessments to make learning more interactive and measurable.

Traditional assessment methods generally depend on printed question papers and manual evaluation. Although these methods are simple to implement, they require additional time and effort for conducting tests, checking answers, and calculating scores. Online quiz systems overcome many of these limitations by automating the evaluation process and displaying results immediately after the quiz is completed.

Several existing online learning platforms provide quiz and assessment features. These systems commonly include multiple-choice questions, automatic score calculation, question navigation, and result generation. Some advanced platforms also provide features such as user accounts, question banks, timers, difficulty levels, performance tracking, and leaderboards. These features demonstrate how web technologies can be effectively used to create interactive educational applications.

Modern web development technologies have made it easier to develop responsive and interactive quiz applications. **HTML** provides the basic structure of the application, while **CSS** is used to create an attractive and responsive user interface. **Java Script** provides dynamic functionality, such as displaying questions, accepting user responses, validating answers, calculating scores, and updating the result dynamically without requiring manual calculations.

A review of existing quiz applications shows that a good quiz system should have a simple interface, clear navigation, properly organized questions, and an efficient result system. User experience is an important factor because complicated navigation or unclear instructions can make the quiz difficult to use. Therefore, the proposed application focuses on maintaining a clean, simple, and user-friendly design.

Existing systems also show that randomization and different quiz categories can improve the usefulness of an online quiz platform. Randomly displaying questions can make each attempt different, while categorizing questions allows users to select topics according to their interests or learning requirements. These concepts can be considered for further development of the proposed system.

From the study of existing systems and web technologies, it can be concluded that a web-based quiz application can provide a convenient and interactive alternative to traditional assessment methods. The proposed **Quiz Application** applies these concepts to create a simple, responsive, and interactive platform for conducting quizzes and evaluating user performance.

3. SYSTEM ANALYSIS AND DESIGN

The System Analysis and Design section explains how the Quiz Application works and what is required to develop it. Before developing the application, the requirements of a typical quiz system were studied. Based on this analysis, the application was designed to allow users to start a quiz, answer questions, view their results, and obtain a certificate.

3.1 System Analysis

System analysis helps to understand the requirements of the Quiz Application and the problems that can occur in a normal quiz system.

Existing System: In a traditional quiz, questions are usually given on paper or the answers are checked manually. The teacher or examiner has to check the answers and calculate the marks. This takes more time, especially when there are many students. It is also difficult to provide the result immediately after completing the quiz.

Problems in the Existing System:

- Answer checking can take a lot of time
- Marks have to be calculated manually
- There can be mistakes while checking answers
- Students cannot see their result immediately
- There may not be a proper time limit for the quiz
- There is usually no automatic certificate generation

Proposed System: To solve these problems, I developed a Quiz Application using HTML, CSS, and JavaScript. In this application, the user first enters their details and selects the quiz topic and number of questions. After starting the quiz, the questions are displayed one by one and the user can select the answers. The application checks the answers automatically and displays the final result after the quiz is completed.

The application also provides a certificate with the student's name, quiz topic, percentage, and grade.

Advantages of the Proposed System:

- Saves time in checking answers
- Automatically calculates the result
- Provides a timer during the quiz
- Makes it easy to move between questions
- Shows correct and wrong answers in the result
- Calculates percentage and grade automatically
- Provides a performance indicator
- Generates a certificate after completing the quiz

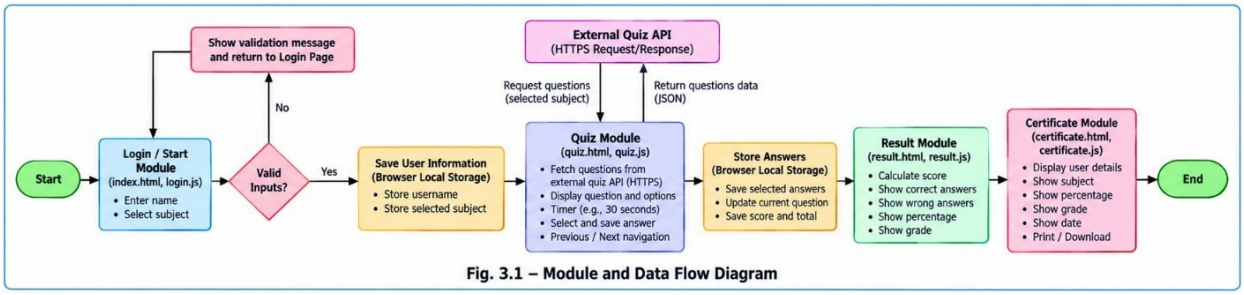


Fig. 3.1 – Module and Data Flow Diagram

3.2 System Design

The system design explains the different parts of the Quiz Application and how they work together.

Modules of the System:

1. **User Details Module**
2. **Quiz Module**
3. **Result Module**
4. **Certificate Module**

User Details Module – This is the first part of the application. The user enters their name, student code, quiz topic, and the number of questions. After entering the details, the user can click on the Start Quiz button.

Quiz Module – This module displays the questions and answer options. It also contains the timer and question navigation buttons. The user can save an answer and move to the next or previous question.

Result Module – After completing the quiz, this module checks the selected answers and calculates the score. It shows the number of correct answers, wrong answers, percentage, and grade.

Certificate Module – This module displays the Certificate of Achievement. It contains the student's name, quiz topic, percentage, grade, and date. The certificate can also be printed or downloaded as a PDF.

System Flow

The working flow of the application is:

Home Page → Enter Details → Start Quiz → Answer Questions → Complete Quiz → Check Answers → Display Result → Generate Certificate

Technologies Used:

● **HTML** – Used to create the structure of all the pages.

● **CSS** – Used to design the pages, buttons, forms, quiz layout, result page, and certificate.

● **JavaScript** – Used to make the quiz interactive and to handle the timer, questions, answers, score calculation, result, and certificate details.

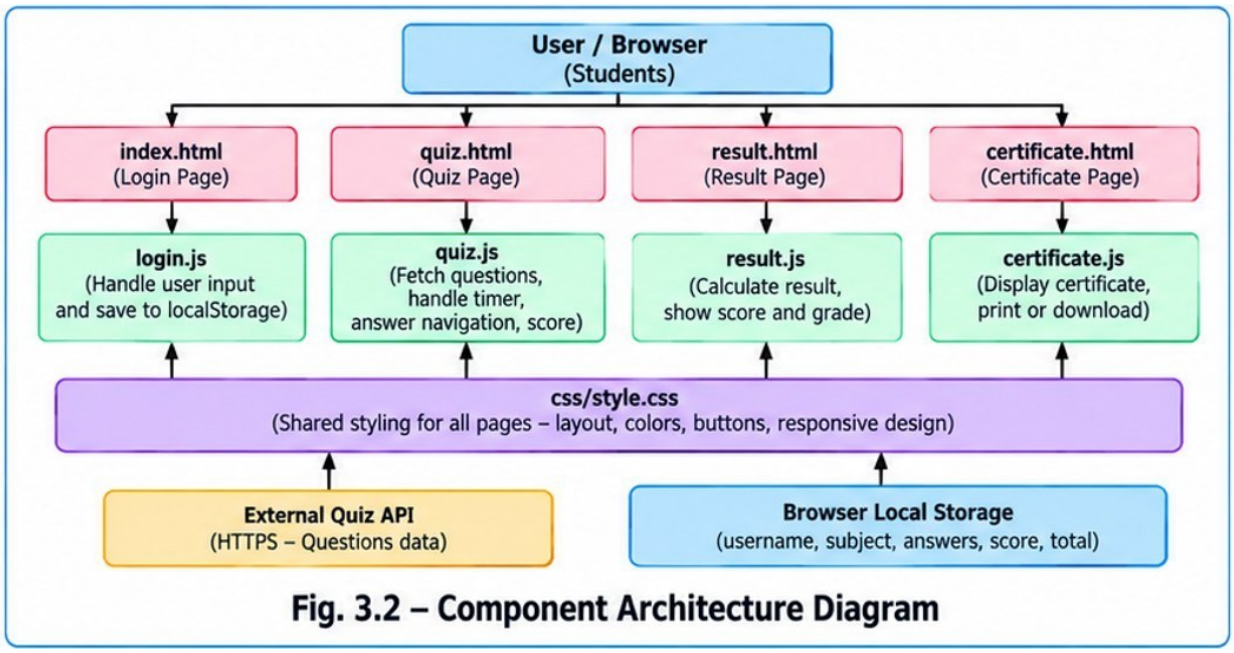


Fig. 3.2 – Component Architecture Diagram

The design of the application is kept simple so that a user can understand and use it without any difficulty. Each part of the application has a specific purpose, starting from entering the details and ending with the final certificate.

4. IMPLEMENTATION

The implementation part explains how I developed the Quiz Application. I used **HTML, CSS, and JavaScript** to develop the complete application.

First, I created the **Home Page** using HTML. The page contains fields for entering the user's name, student code, quiz topic, and number of questions. CSS was then used to give the page a clean design with a purple background, a white form box, rounded corners, and styled buttons.

After the user enters the required details and clicks the **Start Quiz** button, the Quiz Page opens. The Quiz Page displays the questions along with multiple-choice options. JavaScript is used to load the questions and control the quiz.

I added a **timer** using JavaScript so that the user can see the remaining time while attempting the quiz. The question numbers are also displayed on the right side of the page. This makes it easier for the user to know which questions they have to attempt.

The **Previous, Save, and Next** buttons are used for moving between questions. When the user selects an option and saves it, JavaScript keeps track of the selected answer. The user can also go back to a previous question if they want to change their answer.

After all the questions are completed, the application checks the answers using JavaScript. The selected answers are compared with the correct answers stored in the application. The number of correct and wrong answers is then calculated.

JavaScript stores selected user details, answers, and result values in the browser's local Storage so that information can be used across the application's pages. The application also calculates the percentage and assigns a grade based on the user's performance. These details are displayed on the Result Page.

For example, in my output the user answered all 10 questions correctly. Therefore, the result shows:

Score: 10 / 10 Correct Answers: 10 Wrong Answers: 0 Percentage: 100% Grade: A+

A green performance bar is also displayed on the result page to show the overall performance.

After seeing the result, the user can click on **View Certificate**. The Certificate Page then displays the student's name, quiz topic, percentage, grade, and date. In my project, the certificate shows the Java quiz with a score of 100% and an A+ grade.

The certificate page also contains buttons for Print Certificate and Download PDF. The Download PDF button opens the browser print dialog, where the user can select the Save as PDF option.

CSS was used to maintain the same style throughout the application. I used different colors, buttons, borders, spacing, and rounded boxes to make the application look clean and easy to use.

Overall, the implementation was done using basic web technologies. **HTML** was used for the page structure, **CSS** was used for the design, and **JavaScript** was used for the main functionality of the Quiz Application.

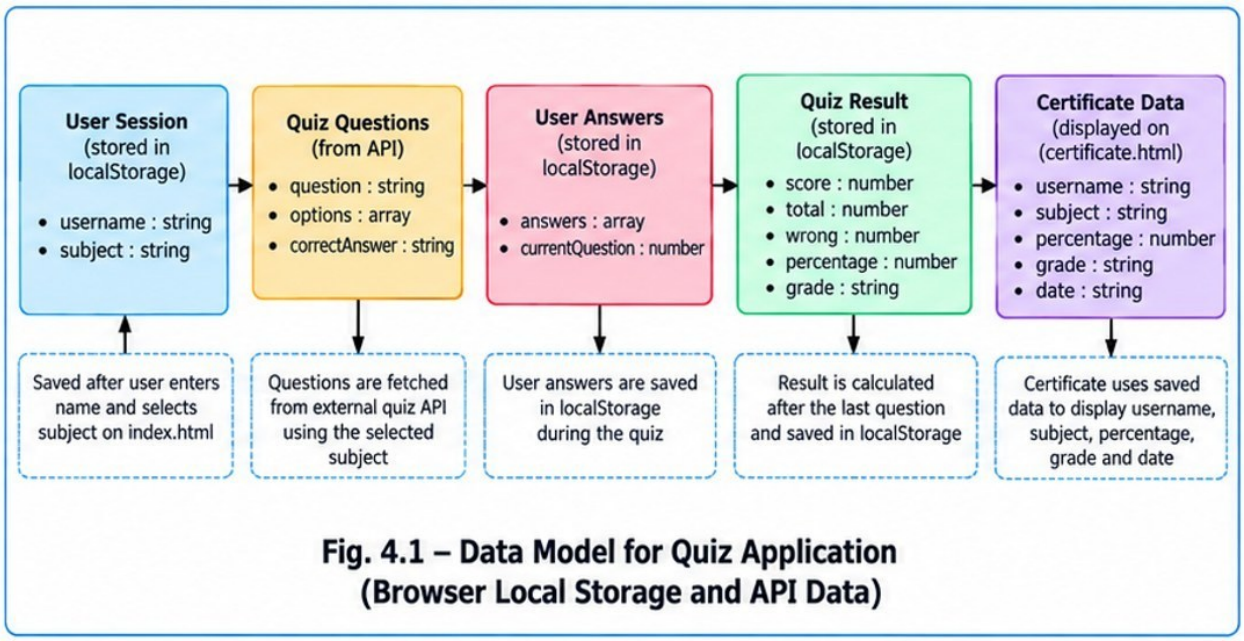


Fig. 4.1 – Data Model for Quiz Application (Browser Local Storage and API Data)

The diagram shows how a quiz application manages user details, fetches questions from an API, stores and calculates results, and generates a certificate based on the quiz performance.

5. OUTPUT SCREENSHOT

The screenshots below show the different pages of the developed Quiz Application. These screenshots demonstrate the complete working flow of the application, starting from entering user information and ending with certificate generation.

5.1 :- Home Page

The Home Page is the starting page of the Quiz Application. It allows the user to enter their full name, student code, quiz topic, and the number of questions.

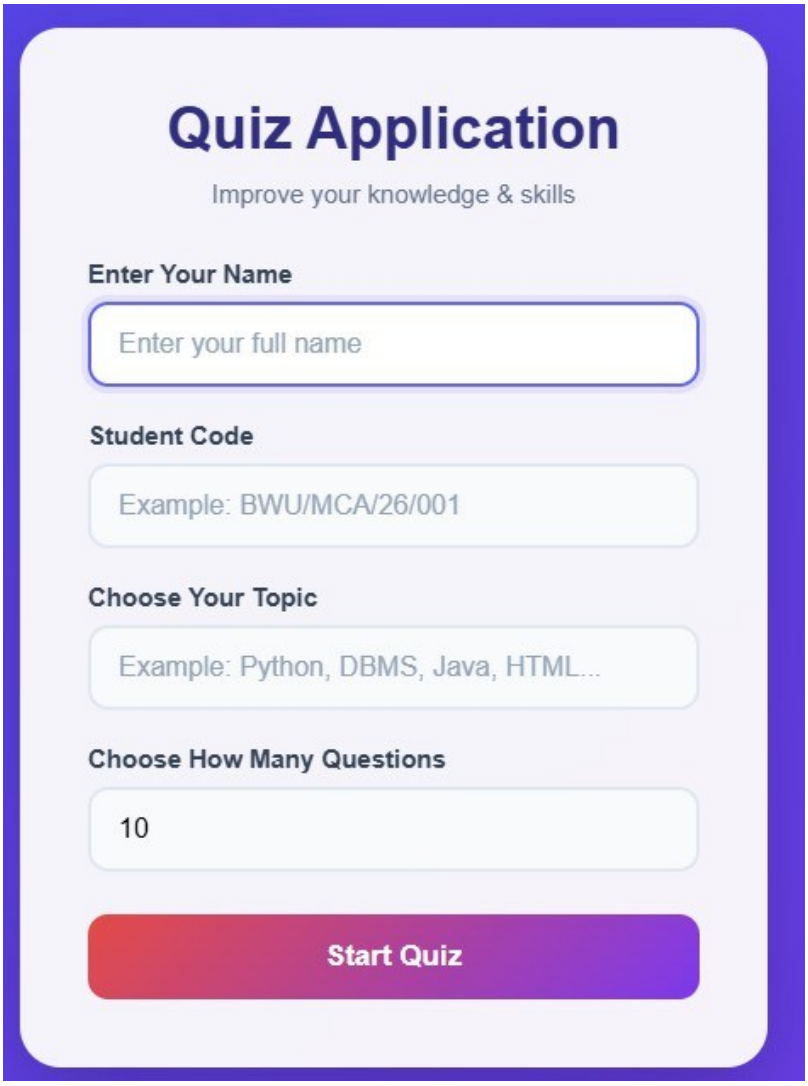


Fig. 5.1 – Quiz Application Home Page

5.2 :- Quiz Page

The Quiz Page is the main page where the user attempts the quiz. It displays the current question, multiple-choice options, remaining time, and question navigation panel.

The user can select an answer and use the Save, Previous, and Next buttons to navigate through the questions. The question navigation buttons allow the user to move between different questions.

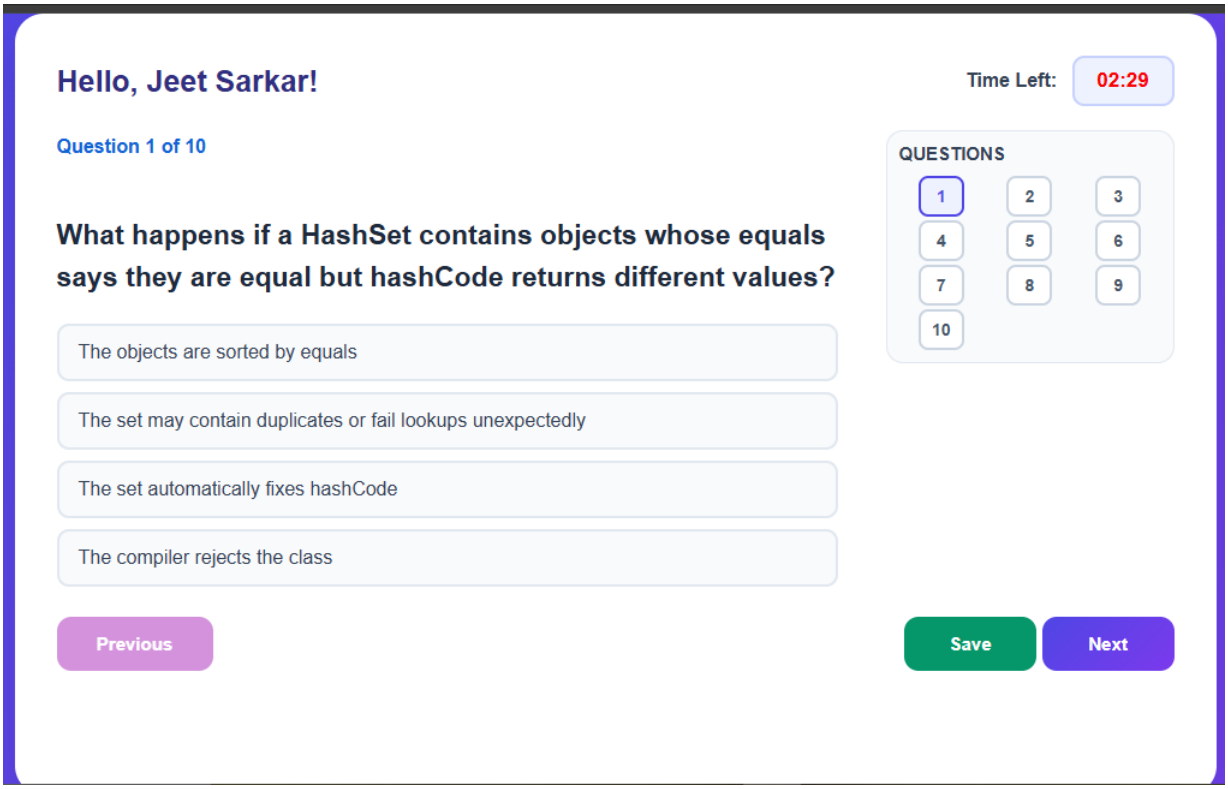


Fig. 5.2 – Quiz Question Page



Fig. 5.3 – Quiz Result and Performance Page



Fig. 5.4 – Certificate of Achievement

6. SOURCE CODE

The following selected source-code excerpts illustrate the main functions of the Online Quiz Application, including input validation, retrieving questions from an external API, displaying questions, answer selection, timing, score calculation, result generation, and certificate actions.

6.1 – Login.js (User Validation and Navigation)

This function validates the user's name and selected subject, saves them in browser local storage, and opens the quiz page.

```
function startQuiz() {
    let username = document.getElementById("username").value;
    let subject = document.getElementById("subject").value;

    if (username === "") {
        alert("Please enter your name.");
        return;
    }
    if (subject === "") {
        alert("Please select a subject.");
        return;
    }

    localStorage.setItem("username", username);
    localStorage.setItem("subject", subject);
    window.location.href = "quiz.html";
}
```

File: js/login.js

6.2 – Quiz.js (Loading Questions from the API)

The application requests up to ten questions from The Trivia API using the selected category. If the request fails or no questions are returned, an error message is shown.

```
async function loadQuiz() {
    try {
        let url =
            "https://the-trivia-api.com/v2/questions?limit=10&categories="
            + subject;
        let response = await fetch(url);

        if (!response.ok) {
            throw new Error("API request failed");
        }

        questions = await response.json();
        if (questions.length === 0) {
            throw new Error("No questions found");
        }

        answers = new Array(questions.length).fill(null);
        showQuestion();
    } catch (error) {
        console.log(error);
        document.getElementById("question").innerText =
            "Unable to load questions.";
    }
}
```

File: js/quiz.js

6.3 – Quiz.js (Displaying Questions and Options)

This function displays the current question, updates the question number and progress text, and creates selectable buttons for answer options.

```
function showQuestion() {
    clearInterval(timer);
    time = 30;
```

```
    document.getElementById("timer").innerText = time;
    startTimer();

    let q = questions[currentQuestion];
    document.getElementById("question-number").innerText =
        "Question " + (currentQuestion + 1) +
        " of " + questions.length;
    document.getElementById("progress").innerText =
        Math.round(((currentQuestion + 1) / questions.length) * 100)
        + "% completed";
    document.getElementById("question").innerText = q.question.text;

    let options = [...q.incorrectAnswers, q.correctAnswer];
    options.sort(() => Math.random() - 0.5);

    let optionsContainer = document.getElementById("options");
    optionsContainer.innerHTML = "";
    options.forEach(option => {
        let button = document.createElement("button");
        button.className = "option";
        button.innerText = option;
        button.addEventListener("click", function () {
            selectAnswer(button, option);
        });
        optionsContainer.appendChild(button);
    });
}
```

File: js/quiz.js

6.4 – Quiz.js (Saving Answers and Navigation)

The selected answer is stored in an array. The Save button stores the answers in local storage, while the navigation functions move between questions.

```
function selectAnswer(button, answer) {
    let buttons = document.querySelectorAll(".option");
    buttons.forEach(btn => btn.classList.remove("selected"));
    button.classList.add("selected");
    answers[currentQuestion] = answer;
}

function saveAnswer() {
    if (answers[currentQuestion] === null) {
        alert("Please select an answer first.");
        return;
    }
    localStorage.setItem("quizAnswers", JSON.stringify(answers));
    alert("Answer saved successfully!");
}

function nextQuestion() {
    clearInterval(timer);
    if (currentQuestion < questions.length - 1) {
        currentQuestion++;
        showQuestion();
    } else {
        finishQuiz();
    }
}

function previousQuestion() {
    if (currentQuestion > 0) {
        clearInterval(timer);
        currentQuestion--;
        showQuestion();
    }
}
```

File: js/quiz.js

6.5 – Quiz.js (Timer and Score Calculation)

Each question has a 30-second timer. When time expires, the application advances. At the end, it calculates the score, saves the score and total question count, and opens the result page.

```
function startTimer() {
    timer = setInterval(() => {
        time--;
        document.getElementById("timer").innerText = time;
        if (time <= 0) {
            clearInterval(timer);
            nextQuestion();
        }
    }, 1000);
}

function finishQuiz() {
    clearInterval(timer);
    let score = 0;
    questions.forEach((question, index) => {
        if (answers[index] === question.correctAnswer) {
            score++;
        }
    });
    localStorage.setItem("score", score);
    localStorage.setItem("total", questions.length);
    window.location.href = "result.html";
}
```

File: js/quiz.js

6.6 – Result.js (Result and Grade Calculation)

The result script retrieves the score and total from local storage, calculates wrong answers and percentage, and assigns a grade.

```
let score = Number(localStorage.getItem("score"));
let total = Number(localStorage.getItem("total"));
let wrong = total - score;
let percentage = Math.round((score / total) * 100);

let grade;
if (percentage >= 90) {
    grade = "A+";
} else if (percentage >= 80) {
    grade = "A";
} else if (percentage >= 70) {
    grade = "B";
} else if (percentage >= 60) {
    grade = "C";
} else if (percentage >= 50) {
    grade = "D";
} else {
    grade = "F";
}

document.getElementById("score").innerText =
    "Score: " + score + " / " + total;
document.getElementById("correct").innerText =
    "Correct Answers: " + score;
document.getElementById("wrong").innerText =
    "Wrong Answers: " + wrong;
document.getElementById("percentage").innerText =
    "Percentage: " + percentage + "%";
document.getElementById("grade").innerText = "Grade: " + grade;
```

File: js/result.js

6.7 – Certificate.js (Certificate Information and Print)

The certificate script calculates the percentage and grade, displays the user's name, subject, percentage, grade and date, and provides print-related actions.

```
document.getElementById("certificate-name").innerText = username;
document.getElementById("certificate-subject").innerText = subjectName;
document.getElementById("certificate-percentage").innerText =
    percentage + "%";
document.getElementById("certificate-grade").innerText  =  grade;

function printCertificate() {
    window.print();
}

function downloadCertificate() {
    window.print();
}

function backToResult() {
    window.location.href = "result.html";
}
```

File: js/certificate.js

Note: The current Download PDF function also opens the browser print dialog using window.print(). The user can select the Save as PDF option in that dialog.

6.8 – HTML Structure (Quiz Page)

The quiz page contains the HTML elements that JavaScript updates dynamically, including the welcome message, timer, progress text, question, options, and navigation buttons.

```
<div class="quiz-header">
    <h2 id="welcome"></h2>
    <div>Time: <span id="timer">30</span></div>
</div>
<div id="progress"></div>
<p id="question-number"></p>
<h2 id="question">Loading question...</h2>
<div id="options"></div>
<div class="quiz-buttons">
    <button id="previous-btn" onclick="previousQuestion()">Previous</button>
    <button id="save-btn" onclick="saveAnswer()">Save</button>
    <button id="next-btn" onclick="nextQuestion()">Next</button>
</div>
```

File: quiz.html

End of Source Code Section

These selected excerpts explain the main implementation. HTML defines the page structure, JavaScript manages quiz behavior and navigation, and local storage passes information between the pages.

7. TESTING AND DEBUGGING

Introduction

Testing and debugging are important stages in the development of a web-based application. Testing is performed to verify that the application works correctly according to the project requirements, while debugging is the process of identifying and correcting errors in the code. These processes help improve the reliability, functionality, and user experience of the application.

In the **Quiz Application** project, testing and debugging were carried out to ensure that the website functions properly, displays questions correctly, accepts user answers, calculates scores accurately, and displays the final results. The application was checked for errors in its structure, design, and functionality.

Types of Testing

1. Functional Testing

Functional testing was performed to verify that the main features of the Quiz Application work as expected. The Start Quiz button, answer selection, question navigation, and result display were checked to ensure that each feature performs its intended function.

2. User Interface Testing

User interface testing was conducted to check the appearance and layout of the application. The colors, fonts, buttons, question sections, spacing, and overall design were examined to ensure that the interface is clear, attractive, and easy to use.

3. Navigation Testing

Navigation testing was performed to ensure that users can move through the quiz without difficulty. The functionality of the Start, Next, Previous, and Submit buttons was checked wherever these features were implemented.

4. Score Calculation Testing

Score calculation testing was carried out to verify that the application calculates the user's score based on the answers submitted. Correct and incorrect answers were tested to confirm that the final score is calculated and displayed accurately.

5. Compatibility Testing

Compatibility checks were carried out in the available web browser environment to review whether the application displayed and operated as expected. The application should also be checked in other commonly used browsers before deployment.

6. Responsive Testing

Responsive layout checks were carried out on the available screen sizes. Additional testing on a wider range of phones, tablets, and desktop displays can help confirm consistent behaviour.

Overall Conclusion

Testing and debugging help ensure that the Quiz Application performs its intended functions correctly and provides a satisfactory user experience. By checking the interface, navigation, answer selection, score calculation, and result display, potential errors can be identified and corrected. These processes improve the overall quality, usability, and reliability of the application.

The testing and debugging phase also provides practical experience in identifying programming errors, correcting code, and maintaining the functionality of a web-based application.

TEST CASES:--

Test Case ID	Input	Test Description	Expected Result	Actual Output	Status
TC01	Application URL	Open the Quiz Application in a browser.	The home page should load successfully.	As expected	Pass
TC02	Click Start Quiz button	Check whether the quiz starts when the button is clicked.	The first question should be displayed.	As expected	Pass
TC03	Display question	Verify that the question and answer options appear correctly.	The question and all answer options should be visible.	As expected	Pass
TC04	Select an answer	Check whether the user can select an answer option.	The selected answer should be registered correctly.	As expected	Pass
TC05	Click Next button	Verify navigation to the next question.	The next question should be displayed.	As expected	Pass
TC06	Click Previous button	Verify navigation to the previous question, if implemented.	The previous question should be displayed.	As expected	Pass
TC07	Submit quiz	Submit the quiz after answering the questions.	The quiz should be submitted successfully.	As expected	Pass
TC08	Correct and incorrect answers	Verify the score calculation.	The score should be calculated according to the answers.	As expected	Pass
TC09	Complete the quiz	Check the final result display.	The final score should be displayed correctly.	As expected	Pass
TC10	Open on a mobile device	Check the application's responsive layout.	The application should remain readable and usable.	As expected	Pass
TC11	Refresh the browser	Check the application's behaviour after refreshing the page.	The application should reload without errors.	As expected	Pass
TC12	Leave an answer unselected	Check how the application handles an unanswered question.	The application should handle the unanswered question according to its design.	As expected	Pass

8. CONCLUSION AND FUTURE ENHANCEMENT

The **Quiz Application** is a web-based project developed as part of the Web Designing course to provide an interactive and user-friendly platform for conducting quizzes and evaluating knowledge. The main purpose of this project is to make the process of answering questions and checking results simple, convenient, and engaging.

The project demonstrates the practical implementation of web development technologies such as **HTML, CSS, and JavaScript**. HTML is used to create the structure of the application, CSS is used to design an attractive user interface, and JavaScript is used to implement interactive features and quiz functionality. The integration of these technologies helps create a functional and visually appealing web application.

The application allows users to participate in quizzes, select answers, navigate through questions, and view their final scores based on their performance, depending on the implemented features. It provides a convenient alternative to traditional paper-based quizzes by reducing the need for manual evaluation and making the assessment process more efficient.

This group project has also provided valuable practical experience in web designing, programming, problem-solving, testing, and debugging. Working collaboratively has helped the team understand the importance of communication, task distribution, coordination, and teamwork in completing a software development project.

Overall, the Quiz Application successfully demonstrates how web technologies can be used to develop an interactive educational tool. The project has helped us improve our technical knowledge and practical skills while understanding the basic process of designing, developing, and testing a web-based application.

Future Enhancement

Although the current Quiz Application provides basic quiz functionality, several additional features can be implemented in the future to improve its usability, performance, and overall functionality.

1. User Registration and Login:
A user authentication system can be introduced to allow users to create accounts, log in securely, and maintain their personal quiz records.

2. Multiple Quiz Categories:
Different quiz categories, such as Computer Science, Web Designing, General Knowledge, Mathematics, and Science, can be added to provide users with a wider range of topics.

3. Advanced Timer Controls:

4. Difficulty Levels:
The application can include easy, medium, and hard difficulty levels so that users can select quizzes according to their knowledge and skill levels.

REFERENCES

MDN Web Docs:
https://developer.mozilla.org
(For JavaScript, HTML and CSS reference)

https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API

https://the-trivia-api.com

(For retrieving multiple-choice quiz questions through the API)

Geeksforgeeks:

https://geeksforgeeks.org
(For basic web development tutorials and suggestions)

W3Schools:
https://www.w3schools.com
(For basic web development tutorials and examples)

Class notes and study materials provided by Brainware University

TOOLS in use:
Visual Studio Code
https://code.visualstudio.com
(For Development environment)

GitHub:
https://github.com
(For version control and collaboration)